

Streamer Memory Leak Fix

Oct 7, 2026 · @Oddne

The data-collection program's memory grew for as long as it ran, because the `sifi_bridge_py` library kept a second, never-read copy of every sensor packet. The fix discards those copies in the project's own streamer code, `CustomLibEMG/Streamer/_sifi_bridge_streamer.py`, without changing the installed library. Memory now stays flat and CPU use fell from about 25% to near zero.

Problem

While streaming from the BioArmband, the program's memory use rose steadily for the whole session and never came back down. CPU use was also high, at about 25%.

An earlier attempt (commit d6d6b03, "Added a reset function to stop progressive memory consumption") cleared the shared-memory buffers. Those buffers have a fixed size, so clearing them could not stop the growth.

Cause

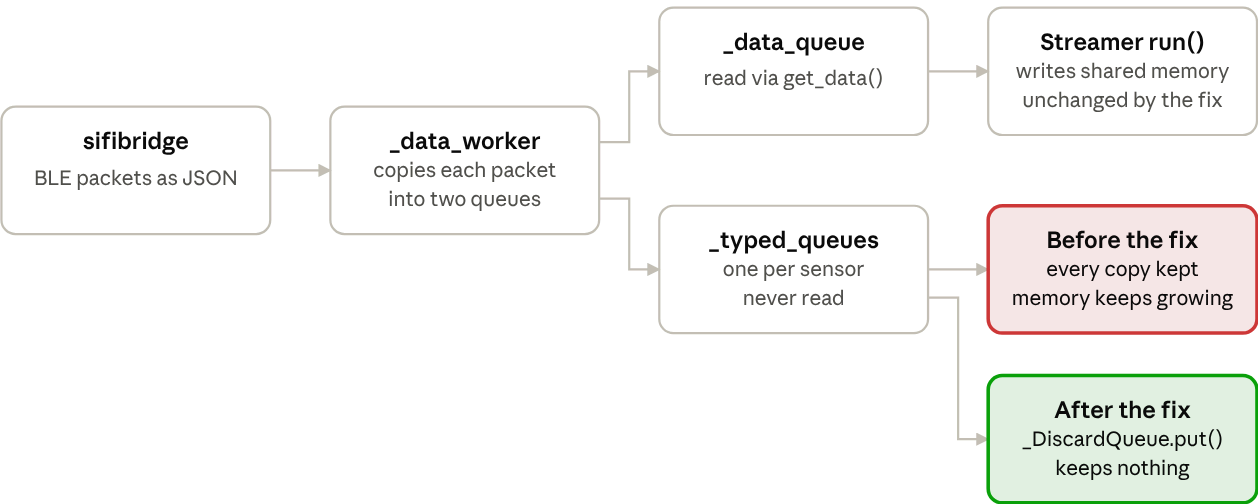
The `sifi_bridge_py` library put every packet it received into two unbounded queues, and the streamer only ever read one of them. The second copies piled up in the streamer process for the whole session.

- **Where it happens:** `SifiBridge._data_worker` in `.venv/lib/python3.12/site-packages/sifi_bridge_py/sifi_bridge.py` (lines 1013–1016). Each packet goes into `_data_queue`, and again into `_typed_queues[sensor]` for its sensor.
- **What reads them:** `SiFiBridgeStreamer.run` reads only `_data_queue`, through `get_data()`. Nothing in the project calls the per-sensor getters such as `get_emg()`, so `_typed_queues` was never drained.
- **Only cleared once:** the library empties the per-sensor queues in `clear_data_buffer()`, which the streamer calls only when it connects.
- **Why it grew fast:** each packet is a dict holding Python lists of floats. It arrives hundreds of times per second across EMG, IMU, ECG, EDA, PPG and temperature.

The high CPU use most likely came from the same leak. Python’s garbage collector repeatedly scanned a growing heap of millions of leaked objects, and the operating system kept handing the process new memory pages. The fix wasn’t aimed at CPU use, and the split between these two costs wasn’t measured.

Every packet went into two queues; only one was ever read

The fix leaves the read path untouched and drops the second copy



packet flow in the streamer process · before and after the fix

The top path is how data reaches shared memory and the CSV files, and the fix leaves it unchanged. Only the second copy, in `_typed_queues` , is now dropped.

Fix

Right after the streamer creates a `SifiBridge` , it swaps that object's per-sensor queues for a dummy queue that accepts packets and keeps nothing. Each packet still reaches `_data_queue` , `get_data()` and shared memory exactly as before. The installed library file is not modified, so reinstalling or updating `sifi-bridge-py` keeps the fix.

All changes are in `CustomLibEMG/Streamer/_sifi_bridge_streamer.py` :

Where	Change
Imports (top of file)	Added <code>import queue</code> ,for <code>queue.Empty</code> .

Where	Change
Module level, after <code>MAX_STDERR_LINES_PER_DRAIN</code>	New class <code>_DiscardQueue</code> : a stand-in for <code>queue.Queue</code> that drops every <code>put</code> and always reads as empty.
Module level, after <code>_DiscardQueue</code>	New function <code>_disable_typed_queues(sb)</code> : replaces every entry of <code>sb._typed_queues</code> with a <code>_DiscardQueue</code> .
<code>SiFiBridgeStreamer.run()</code>	Calls <code>_disable_typed_queues(self.sb)</code> right after the first <code>sbp.SifiBridge()</code> is created, before <code>connect()</code> .
<code>SiFiBridgeStreamer._rebuild_bridge()</code>	Same call after the replacement <code>sbp.SifiBridge()</code> , so a reconnected bridge is covered too.

The dummy queue implements `get_nowait()` , `get()` , `qsize()` and `empty()` , so the library's `clear_data_buffer()` and per-sensor getters still work against it. If a future library version drops the private `_typed_queues` attribute, the streamer prints a one-line warning instead of crashing. Memory would then start growing again until the attribute name is updated.

Code

The code below is in `CustomLibEMG/Streamer/_sifi_bridge_streamer.py` and was committed in cc5436b ("Test memory consumption fix").

The dummy queue and the helper, at module level (lines 70–113):

```
class _DiscardQueue:
    """Stand-in for a queue.Queue that keeps nothing.

    Accepts puts and always reads as empty, so sifi_bridge_py's
    clear_data_buffer() and get_<sensor>() calls still work against it.
    """

    def put(self, item, block=True, timeout=None):
        pass
```

```

def put_nowait(self, item):
    pass

def get(self, block=True, timeout=None):
    raise queue.Empty

def get_nowait(self):
    raise queue.Empty

def qsize(self):
    return 0

def empty(self):
    return True

def _disable_typed_queues(sb):
    """Stop a SifiBridge from keeping a second copy of every packet.

    sifi_bridge_py puts each packet into the generic queue read by get_data()
    and, again, into an unbounded per-sensor queue meant for get_emg() etc.
    This streamer only uses get_data(), so nothing ever drains the per-sensor
    queues and they grow for as long as the stream runs -- the slow memory
    climb of the streamer process. Swapping them for discarding queues on our
    own instance fixes that without modifying the installed library.
    """
    typed_queues = getattr(sb, "_typed_queues", None)
    if not isinstance(typed_queues, dict):
        print(
            "LibEMG -> SiFiBridgeStreamer (sifi_bridge_py has no\n\
_typed_queues; "\n\
            "could not disable the per-sensor queues, memory may grow over\n\
time)."\n\
        )
        return
    for sensor in typed_queues:
        typed_queues[sensor] = _DiscardQueue()

```

The two call sites, in `_rebuild_bridge()` (line 530) and `run()` (line 811):

```

# _rebuild_bridge(): the replacement bridge after a dropped connection
self.sb = sbp.SifiBridge()
self.sb._DEFAULT_REQUEST_TIMEOUT = 10.0
_disable_typed_queues(self.sb)

# run(): the first bridge, before connect()
self.sb = sbp.SifiBridge()
self.sb._DEFAULT_REQUEST_TIMEOUT = 10.0
_disable_typed_queues(self.sb)

self.connect()

```

At the top of the file, `import queue` was added beside the existing imports.

Verification

The fix was checked offline first, then by running the program with the armband.

Offline test. 5000 packets were fed through the library's real `_data_worker` over a local socket, with no armband or `sifibridge` binary:

Run	Packets read through <code>get_data()</code>	Copies left in per-sensor queues
Without the fix	5000	5000
With the fix	5000	0

`clear_data_buffer()` still ran without errors against the dummy queues.

With the armband. After the fix the program no longer grows in memory over a session. CPU use dropped from about 25% to near zero. The EMG Monitor's packet check still showed the full 2000 Hz rate, so the streamer is doing the same work with less overhead.

The planned fallback of clearing the `sifibridge` binary's internal buffer every 60 s was not needed, because memory stayed flat without it.